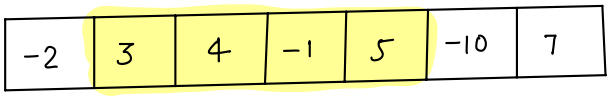
Lecture :- Arrays - 1

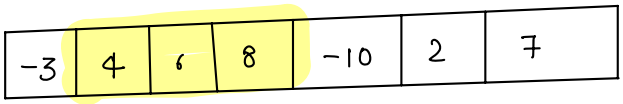
Agenda

- Kadane's Algorithm — pattern subarray [max sum]
- Beggars outside temple
- Merge overlapping intervals.

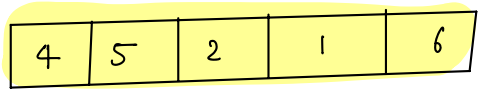
Qu-1 find max subarray sum.



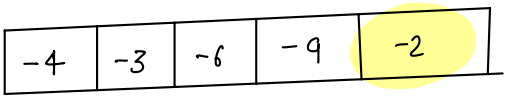
11 Ans



18 Ans



18 Ans



-2 Ans

Brute force approach

```
int maxSubArraySum(int[] A) {
    ans = -∞;
    for(i=0; i<n; i++) {
        for(j=i; j<n; j++) {
            subarray [i, j]
            sum = 0
            for(k=i; k<=j; k++) {
                sum += A[k];
            }
            ans = Math.max(ans, sum);
        }
    }
    return ans;
}
```

TC: $O(n^3)$
SC: $O(1)$

Approach 2

```
int maxSubArraySum(int[] A) {
    pf[] = getPrefixArray(A);
    ans = -∞;
    for(i=0; i<n; i++) {
        for(j=i; j<n; j++) {
            subarray [i, j]
            sum = 0
            if(i==0) {
                sum = pf[j];
            } else {
                sum = pf[j] - pf[i-1];
            }
            ans = max(ans, sum);
        }
    }
    return ans;
}
```

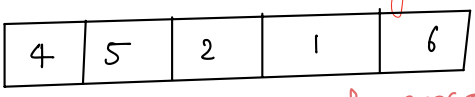
sum[i, j] ⇒
- pf[j] - pf[i-1]
- i == 0 → pf[j]

TC: $O(n^2)$
SC: $O(n)$

Approach 3

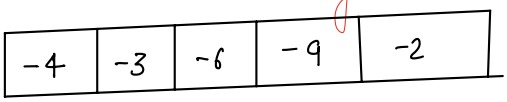
TC: O(n) SC: O(1)

case1 :- All no. in array are +ve.



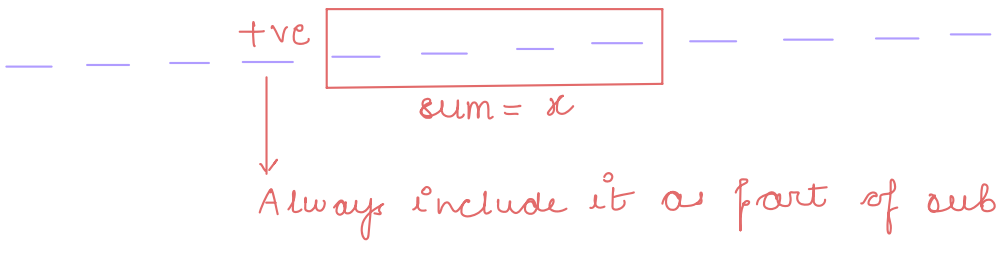
Ans = sum of array ⇒ 18

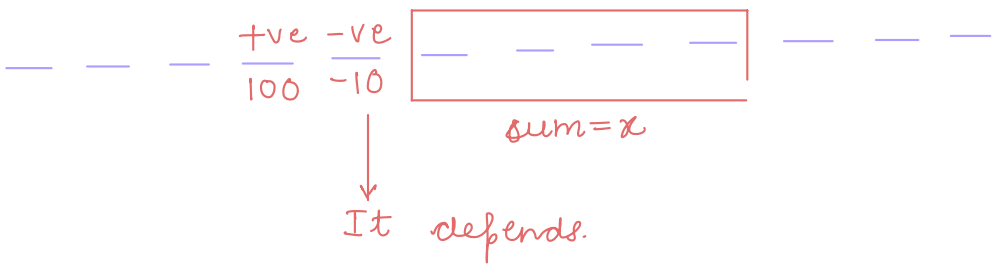
case2: All no in array are -ve.



Ans = Max value of array ⇒ −2

case3 Some no. are +ve and -ve





A[] =	5	6	7	-3	2	-10	-12	8	12	21	-4	7
sum [0]	5	11	18	15	17	7	~~5~~ 0	8	20	41	37	44
max -∞	5	11	18	18	18	18	18	18	20	41	41	~~44~~

A[] =	-20	-10	-6	-15	-2	-30
sum =						
max =						

A[] =	-2	3	4	-1	5	-10	7	
sum [0]	~~-2~~ 0	3	7	6	11	1	8	
max (-∞)	-2	3	7	7	11	11	11	

Algorithm

Kadane's algorithm

```
int maxSubArraySum(int[] A) {
    sum = 0
    ans = -∞
    for(int el: A) {
        sum += el;
        ans = max(ans, sum);
        if(sum < 0) {
            sum = 0;
        }
    }
    return ans;
}
```

TC: O(n)

SC: O(1)

Beggars outside temple { Amazon, Phonepe }

Given arr[n]. All elements are zero initially.

Given Q queries { index, value }.

Add this value starting from 'idx' till end of array.

Queries = 4

idx	value
1	3
4	2
2	1
1	-1

arr ⇒

0	1	2	3	4	5	6
0	0	0	0	0	0	0
0	3	3	3	3	3	3
0	3	3	3	5	5	5
0	3	4	4	6	6	6
0	2	3	3	5	5	5

Ans

Brute force: TC: O(Q * n)

SC: O(1)

Expected TC: Linear TC

Intuition:

	0	1	2	3	4	5	6
A =	0	0	x	0	0	0	0

pf[] ⇒			x	x	x	x	x

idx	value
1	3
4	2
2	1
1	-1

	0	1	2	3	4	5	6
	0	0	0	0	0	0	0
	0	3	0	0	0	0	0
	0	3	0	0	2	0	0
	0	3	1	0	2	0	0
	0	2	1	0	2	0	0
pf:	0	2	3	3	5	5	5

Algorithm

```
void beggarsOutsideTemple(int[] A, int[][] queries) {
    n = A.length;
    q = queries.length
    for(i=0; i<q; i++) {                    ---------- Q
        idx = queries[i][0]
        val = queries[i][1]
        A[idx] += val;
    }
    Create prefix array
    for(i=1; i<n; i++) {                    ---------- n
        A[i] = A[i-1] + A[i];
    }
}
```

TC: O(n+Q)
SC: O(1)

Break: 10:27 – 10:37

Extension of Qu. 2

Given arr[n]. All elements are zero initially.

Given Q queries { start index, end index, value }

Add this value starting from 'idx' till end of array.

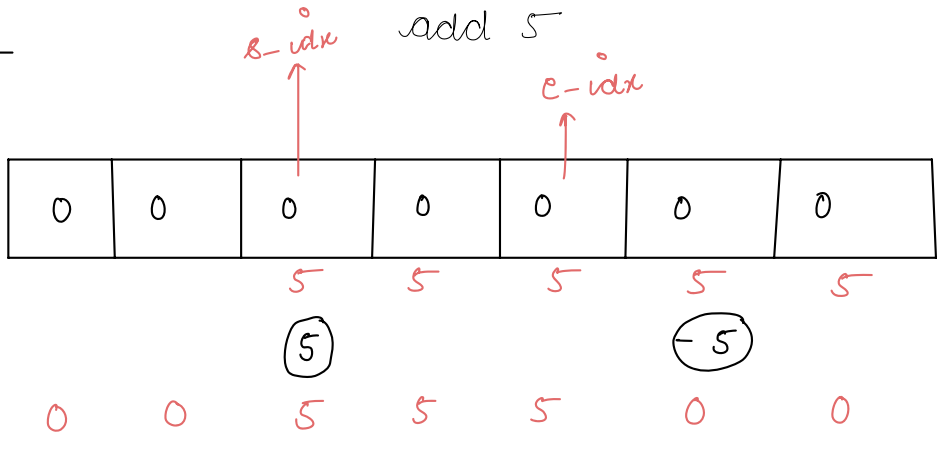
s_idx	e_idx	value
2	4	4
1	3	1
0	2	3
3	5	4

0	1	2	3	4	5
0	0	0	0	0	0
0	0	4	4	4	0
0	1	5	5	4	0
3	4	8	5	4	0
3	4	8	9	8	4

i	j	val
1	4	3
0	5	-1
2	2	4
4	6	3

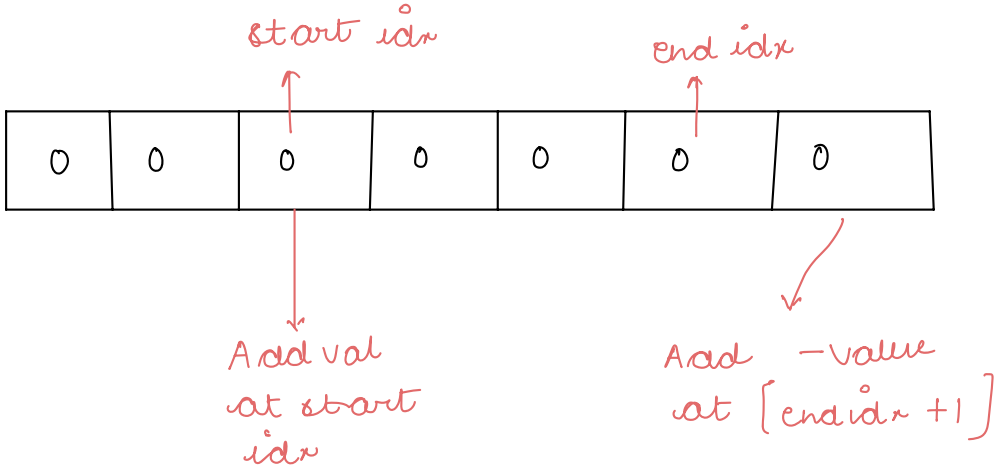
0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0
0	3	3	3	3	0	0	0
-1	2	2	2	2	-1	0	0
-1	2	6	2	2	-1	0	0
-1	2	6	2	5	2	3	0

Approach

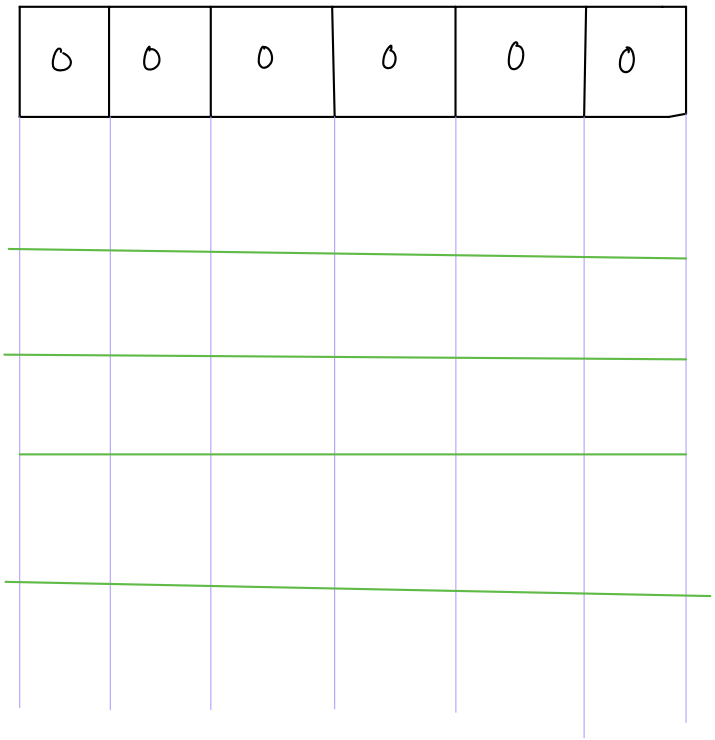


Previous approach ⇒

My arr: 0 0 5 5 5 0 0



i	j	value
2	4	2
1	3	1
0	2	3
3	5	4



Algorithm

```
void beggarsOutsideTemple(int[] A, int[][] queries) {

    n = A.length;
    q = queries.length
    for(i=0; i<q; i++) {                                  ——— Q
        s_idx = queries[i][0]
        e_idx = "    [i][1]
        val = "    [i][2]
        A[s_idx] += val
        if (e_idx+1 < n) {
            A[e_idx+1] -= val;
        }
    }

    Create prefix array
    for(i=1; i<n; i++) {                                  ——— n
        A[i] = A[i-1] + A[i];
    }
}
```

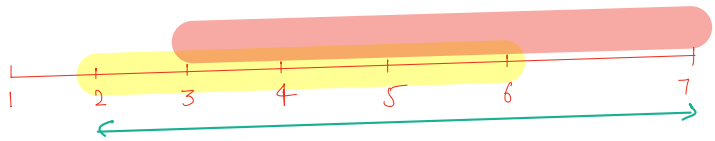
TC: O(n+Q)
SC: O(1)

Merge intervals

interval is defined by start and end time.

start <= end time.

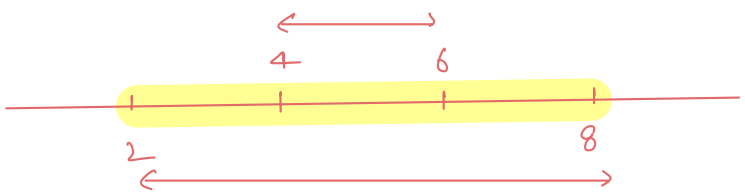
1.) Interval 1 (2, 6) Interval 2 (3, 7) overlapping



final merged interval ⇒ [2, 7]

2.) (2, 4) (5, 7) ⟶ Non-overlapping

3.) (2, 8) (4, 6) ⟶



final merged interval ⇒ [2, 8]

4.) (3, 9) (1, 6) ⟶ final : [1, 9]

Generalisation

[s1, e1] [s2, e2] —— overlapping

e1 can be equal to s2

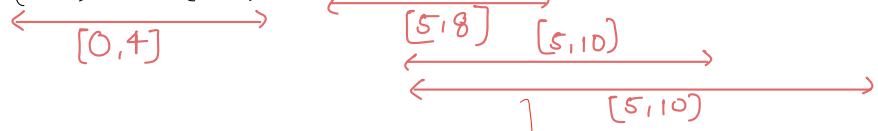
After merging them —

[min(s1, s2), max(e1, e2)]

Qu. Merge sorted overlapping intervals.

Given n intervals in sorted manner [overlapping intervals sorted on start time]. Merge all overlapping intervals sorted on start time and return sorted list.

Input [(0, 2) (1, 4) (5, 6) (6, 8) (7, 10) (8, 9) (12, 14)]



output [(0,4) [5,10] [12, 14]] Ans

| (1,10) (2,3) (4,5) (9,12) |

(1,10)

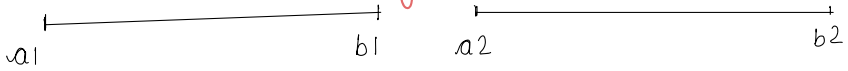
(1,10)

(1,12)

final : [1,12] Ans

Approach Overlapping condition ?

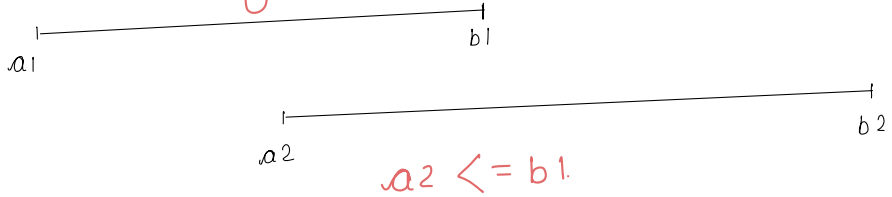
case1 Non-overlapping



$a2 > b1$

$(start)_{i_2} > (end)_{i_1}$

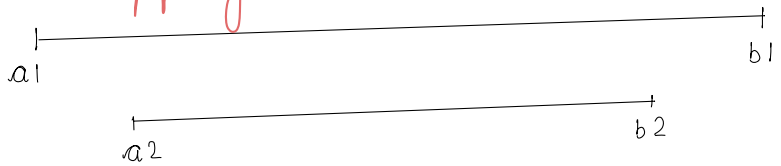
case2: Overlapping



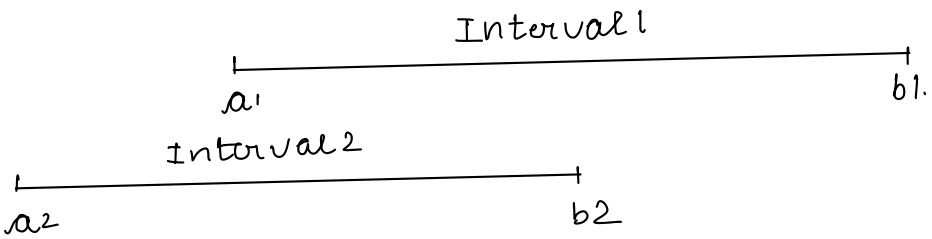
$a2 <= b1$

$(start)_{i_2} <= (end)_{i_1}$

case3: Overlapping



Case4 Invalid case



Dry run:

interval[] = [(0,2) (1,4) (5,6) (6,8) (7,10) (8,9) (12,14)]

(0,4) (5,8) (5,10) (5,10)

interval1	interval2	isoverlapping?	After merging answer list.
0,2 a1 b1	1,4 a2 b2	a2 <= b1.	
0,4 a1 b1	5,6 a2 b2	a2 > b1. X	[0,4]
(5,6) a1 b1	(6,8) a2 b2	a2 <= b1 6 <= 6	
5,8 a1 b1	7,10 a2 b2	a2 <= b1	
5,10 a1 b1	8,9 a2 b2	a2 <= b1	
(5,10) a1 b1	(12,14) a2 b2	a2 > b1	[0,4][5,10][12,14]

One interval must be added at end

Ex: { [2,8] [5,12] [10,16] }

(2,16)

(2,12)

i1	i2	isoverlapping	Merging
2,8	5,12	Yes	
(2,12)	(10,16)	Yes	

Algorithm code

```
class Interval {
    int start;
    int end;
}

List<Interval> mergeOverlappingIntervals(List<Interval> intervals) {
    List<Interval> ans;
    a1 = intervals.get(0).start;
    b1 = "        .get(0).end;
    for(i=1; i<n; i++) {
        a2 = intervals.get(i).start;
        b2 = "        .get(i).end;
        if (a2 > b1) {
            temp = new Interval(a1, b1)
            ans.add(temp);
            a1 = a2;
            b1 = b2;
        }
        else {
            a1 = min(a1, a2)
            b1 = max(b1, b2);
        }
    }
    temp = new Interval(a1, b1)
    ans.add(temp);
    return ans;
}
```

Non-overlapping

Overlapping

It is not wrong but can be avoided

TC: O(n)
SC: O(n)

Thank you :)

kadane's variation

To figure subarray

```
start = 0
end = 0

curstart = 0
maxsum = A[0]
maxEndingHere = A[0]
i = 1 ---------- n
    if (maxEndingHere + A[i] < A[i])
    {
        maxEndingHere += A[i]
        curstart = i
    }
    else {
        maxEndingHere += A[i]
    }
    if (maxEnding > maxsum) {
        maxsum = maxEnding
        start = curstart
        end = i
    }
}
```